CHRONO SUPPORT FOR HUMAN-IN-THE-LOOP SIMULATION

Speaker: Kyle Sha
(Simulation-Based Engineering Lab)

Overview of the human-in-the-loop tutorial

- Human-in-the-loop = the simulation runs at wall-clock speed and a person closes the control loop
- In this tutorial, you will learn how to
  - PART 1: Keep a Chrono::Vehicle simulation real-time and measure how well you did
  - PART 2: Put a human on the keyboard with ChInteractiveDriver
  - PART 4: Bring a device Chrono doesn't know about, and close the loop back to it
  - Parts 3 and 5 (gamepad/wheel, vehicle picker, overlay flags) are in the repo - no time today
- Everything runs in PyChrono; no special hardware is required
- The whole human-in-the-loop contract is: spin once per step, three floats in, state out

Before we start ……

- This is a walkthrough - nothing to install right now; everything runs on my machine
- Code, slides and a written walkthrough: github.com/ksha23/chrono-hil-tutorial
- To run it yourself afterward:
  - `conda create -n chrono_hil -c projectchrono -c conda-forge pychrono pygame python=3.13`
  - `conda activate chrono_hil`
- Do NOT `pip install pychrono` - the PyPI package with that name is an unrelated library
- python tutorial_HIL_driver.py opens an Irrlicht window with an HMMWV - drive it with the arrow keys
- All switches are in the CONFIGURATION section at the bottom of tutorial_HIL_driver.py

Part 1: Keep the simulation real-time

What "real-time" means in Chrono

- Chrono integrates as fast as it can; it has no notion of wall-clock time by itself
- Real Time Factor: RTF = compute time for a step / step size
  - RTF < 1: faster than real time - we must WAIT after each step
  - RTF > 1: slower than real time - nothing can be done except a cheaper model or a bigger step
- Soft real-time: spin after each step until wall time catches up with simulation time
  - No OS scheduling guarantees; good enough for a driving simulator, not for a control ECU
- Three equivalent mechanisms in PyChrono (next slides): `ChRealtimeStepTimer`, `ChVehicle.EnableRealtime`, or your own

Step 1: The simulation loop (Lines 376 to 404)

```
while vis.Run():
    sim_time = system.GetChTime()

    # Render scene
    if step_number % render_steps == 0:
        vis.BeginScene()
        vis.Render()
        vis.EndScene()

    # PART 4: sample the device ONCE per step and hold it for the step
    if device is not None:
        smoother.set_target(*device.poll())
        smoother.advance(step_size)
        smoother.apply_to(driver)

    # Get driver inputs (three floats) - this is the whole human-in-the-loop contract
    driver_inputs = driver.GetInputs()

    # Update modules (process inputs from other modules)
    driver.Synchronize(sim_time)
    terrain.Synchronize(sim_time)
    vehicle_model.Synchronize(sim_time, driver_inputs, terrain)
    vis.Synchronize(sim_time, driver_inputs)

    # Advance simulation for one timestep for all modules
    driver.Advance(step_size)
    terrain.Advance(step_size)
    vehicle_model.Advance(step_size)  # spins here if REALTIME == "vehicle"
    vis.Advance(step_size)
```

Synchronize: every module reads what it needs from the others at time t

Advance: every module integrates from t to t + step

The driver is queried BEFORE Synchronize - inputs are sampled once and held for the step

Step 2: Measure it - sim time vs wall time (Lines 406 to 412)

```
# Console report: how far is the sim from wall time?
if step_number % report_steps == 0:
    wall = time.perf_counter() - wall_start
    print(f"{sim_time:7.2f} {wall:7.2f} {wall - sim_time:+7.3f} {vehicle.GetRTF():6.2f}"
          f" {vehicle.GetSpeed():7.2f}  {driver_inputs.m_steering:6.2f}"
          f" {driver_inputs.m_throttle:5.2f} {driver_inputs.m_braking:5.2f}")
```

drift = wall time - simulation time: negative means the sim is ahead of the clock

vehicle.GetRTF() is the true RTF of the last step, even when real time is enforced

Show Time

- REALTIME = "none": watch the drift column run away
- Change step_size to 5e-4 - RTF goes above 1 and the sim can never be real-time

Step 3: Enforce real time, three ways (Lines 357 to 362)

```
# Real-time setup (PART 1)
# ------------------------------------------------------------------
if REALTIME == "vehicle":
    vehicle.EnableRealtime(True)
rt_timer = chrono.ChRealtimeStepTimer()  # used when REALTIME == "per_step"
cum_timer = CumulativeRealtimeTimer()    # used when REALTIME == "cumulative"
```

"vehicle": ChVehicle::EnableRealtime(True) spins inside vehicle.Advance() using a ChRealtimeStepTimer
"per_step": the same timer, called explicitly at the end of the loop (next slide)
"cumulative": our own timer that compares total simulated time to total wall time

Step 4: A drift-free timer in 10 lines (Lines 56 to 74)

```
class CumulativeRealtimeTimer:
    """Soft real-time that does not accumulate drift.

    ChRealtimeStepTimer measures each step independently: any time lost on a
    slow step (or spent in Python between steps) is never recovered.  This timer
    instead compares the *total* simulated time with the *total* wall time, so a
    slow step is followed by a few fast steps that catch back up.
    """

    def __init__(self):
        self.reset()

    def reset(self):
        self.t_start = time.perf_counter()

    def spin(self, sim_time):
        while time.perf_counter() - self.t_start < sim_time:
            pass  # spin in place until real time catches up
```

Compare TOTAL simulated time with TOTAL wall time - lost time is caught up on the next fast steps
This is how driving simulators (e.g. Chrono::HIL) keep their clock; the same idea also works across processes

Show Time

- Measured on a laptop, HMMWV + TMeasy tires, step 3 ms (RTF ~ 0.14):
  - "none": drift -69 s after 17 s of wall time
  - "per_step": drift +0.012 s after 17 s and growing
  - "vehicle": drift +0.021 s after 17 s and growing
  - "cumulative": drift +0.000 s
- Drift matters more the closer you are to RTF = 1

Part 2: A human on the keyboard

Step 1: ChInteractiveDriver (Lines 296 to 307)

```
elif INPUT_SOURCE == "keyboard":
    ### PART 2: keyboard through the Irrlicht window ###
    # Arrow keys steer/accelerate/brake, 'C' centers steering, 'R' releases
    # the pedals, 'L' locks the current inputs.
    driver = veh.ChInteractiveDriver(vehicle)
    steering_time = 1.0  # time to go from 0 to +1 (or from 0 to -1)
    throttle_time = 1.0  # time to go from 0 to +1
    braking_time = 0.3   # time to go from 0 to +1
    driver.SetSteeringDelta(render_step_size / steering_time)
    driver.SetThrottleDelta(render_step_size / throttle_time)
    driver.SetBrakingDelta(render_step_size / braking_time)
    driver.SetGains(4.0, 4.0, 4.0)  # first-order lag from key target to applied input
```

Arrow keys steer/accelerate/brake, C centers steering, R releases the pedals, L locks the inputs
Each keypress moves a TARGET by "delta"; the applied input follows the target with a first-order lag (SetGains)
delta = render_step / time-to-full-scale: full steering lock takes 1 s, full braking 0.3 s

Step 2: Route the window events to the driver (Lines 339 to 342)

```
vis.AddSkyBox()
vis.AttachVehicle(vehicle)
if INPUT_SOURCE in ("keyboard", "gamepad"):
    vis.AttachDriver(driver)  # route Irrlicht key/joystick events to the driver
```

The Irrlicht visual system owns the keyboard; AttachDriver forwards key events to the driver
Nothing else in the loop changes - the driver still just returns three floats
The same driver reads a gamepad or wheel natively - SetInputMode(JOYSTICK), presets ship with Chrono

Show Time

- Driving the HMMWV with the arrow keys - watch the Irrlicht HUD (speed, inputs, RTF)

Part 4: A device Chrono doesn't know about, and closing the loop

a UDP console, a gamepad, a steering wheel, a phone, another process…

Part 4's architecture: two processes, one UDP contract

→ steering, throttle, braking

Operator machine
operator_console.py

Simulation machine
tutorial_HIL_driver.py

speed, RTF, lateral accel ←

Both directions are plain UDP datagrams - two processes, possibly two machines, no shared memory. The grey arrow only fires when SEND_FEEDBACK = True.

The human-in-the-loop contract

- A driver is just three floats per step: steering [-1, 1], throttle [0, 1], braking [0, 1]
- `veh.ChDriver` is the plain container: `SetSteering()`, `SetThrottle()`, `SetBraking()`
- So the recipe for ANY device is:
  - 1. poll the device (never block the physics loop)
  - 2. rate-limit / smooth (a human arm is not a step function)
  - 3. write into the ChDriver once per step - zero-order hold until the next step
- Today: a UDP operator console in a second terminal - the operator side has no Chrono dependency at all

Step 1: A UDP input source (Lines 136 to 169)

```
class UdpInput:
    """Receive 'steering,throttle,braking' text datagrams from an operator.

    Run operator_console.py in another terminal (or on another machine) and
    drive with the arrow keys there.  The socket is non-blocking: the physics
    loop never waits for the operator.  If no packet arrived since the last
    step, the previous target is held (zero-order hold).
    """

    def __init__(self, port=9870):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(("0.0.0.0", port))
        self.sock.setblocking(False)
        self.operator_addr = None
        self.last = (0.0, 0.0, 0.0)
        self.packets = 0
        print(f"[udp] listening on port {port} - start operator_console.py")

    def poll(self):
        # Drain everything that is queued and keep only the newest packet
        while True:
            try:
                data, addr = self.sock.recvfrom(256)
            except BlockingIOError:
                break
            try:
                s, t, b = (float(x) for x in data.decode().split(","))
                self.last = (s, t, b)
                self.operator_addr = addr
                self.packets += 1
            except ValueError:
                pass
        return self.last
```

Non-blocking socket, drained every step; the newest packet wins, an empty queue holds the last value
The operator can be anywhere on the network - and the console has no Chrono dependency at all

Step 2: Sample once per step, then hold (Lines 384 to 393)

```
# PART 4: sample the device ONCE per step and hold it for the step
if device is not None:
    smoother.set_target(*device.poll())
    smoother.advance(step_size)
    smoother.apply_to(driver)

# Get driver inputs (three floats) - this is the whole human-in-the-loop contract
driver_inputs = driver.GetInputs()
```

poll -> smooth -> write, then GetInputs() as before; the physics loop never waits on the device

Step 3: Send state back at render rate (Lines 413 to 417)

```
# PART 4: feedback to the operator (speed, RTF, lateral acceleration, applied inputs)
if SEND_FEEDBACK and device is not None and step_number % render_steps == 0:
    acc = vehicle.GetPointAcceleration(chrono.ChVector3d(0, 0, 0))
    device.send_feedback(f"{sim_time:.3f},{vehicle.GetSpeed():.3f},{vehicle.GetRTF():.3f},{acc.y:.3f},"
                         f"{driver_inputs.m_steering:.3f},{driver_inputs.m_throttle:.3f},{driver_inputs.m_braking:.3f}")
```

Speed, RTF and lateral acceleration go back to whoever sent us inputs (UdpInput remembers the address)

Send at render rate (50 Hz), not at every physics step - the human cannot use 333 Hz

Show Time

- Two terminals on one laptop over localhost - or two machines over the network, same code either way
- SEND_FEEDBACK = True: speed, RTF and lateral acceleration come back to the console while you drive
- Stop the console mid-drive: the last input is held - what should a real simulator do here?

Where this goes next

- Chrono::HIL (github.com/zzhou292/chrono-HIL): the same three-float interface, scaled up
  - SDL steering wheels with force feedback, cumulative real-time timer, multi-vehicle traffic
  - ROS 2 and Unity bridges, teleoperation over the network, deformable (SCM) terrain
- Chrono::Sensor: cameras/lidar rendered from the vehicle for the operator's screen
- Chrono::Synchrono: several humans in the same world, each on their own machine
- Takeaways
  - Real time in Chrono is soft: spin once per step, and measure the drift
  - The human interface is three floats per step, sampled once and smoothed
  - Never let the input device block the physics loop

Any questions?

Thank you.

kasha2@wisc.edu
Simulation-Based Engineering Lab
University of Wisconsin - Madison

Lab website: http://sbel.wisc.edu
Chrono website: http://www.projectchrono.org
Source code: https://github.com/projectchrono
Tutorial code + slides: https://github.com/ksha23/chrono-hil-tutorial